

Simulador de blockchain PoW/PoS:

fundamentación matemática de las decisiones de diseño

Santiago Cosme

Universidad Anáhuac México — Facultad de Ingeniería

3 de octubre de 2026

1. Introducción y alcance

Este reporte documenta las decisiones de diseño del simulador de blockchain construido para el examen de MT, que implementa **Proof of Work (PoW)** y **Proof of Stake (PoS)** como dos modos de consenso intercambiables sobre el mismo formato de bloque y de transacción, con N nodos configurables ($10 \leq N \leq 20$), cada uno con su propio par de llaves y su propia copia de la cadena. La guía del proyecto sugiere Python + Flask; por decisión explícita se implementa la misma especificación sobre el stack ya existente del proyecto (C++17, PostgreSQL, React/TypeScript), que es funcionalmente equivalente para los fines de esta evaluación: ambos son servidores HTTP con un núcleo de consenso separable de la capa web, tal como exige la sección 6 de la guía.

El dominio de la aplicación (un *ledger* de responsabilidad sobre trabajo de agentes de IA) se conserva como la fuente de las transacciones que entran a los bloques; no se agregó un sistema de transferencias emisor/receptor/monto con saldo transferible, por las razones que se justifican en la Sección 5.

2. Estructura común a ambas versiones

Un bloque se serializa de forma canónica (orden fijo de campos, separados por `|`) antes de hashear, para que el hash no dependa del orden de inserción en una estructura de datos:

$$H(b) = \text{SHA256}\left(\text{SHA256}\left(v1|altura|hash_anterior|merkle|timestamp|proponente|nonce|dificultad\right)\right)$$

El doble SHA-256 (en vez del SHA-256 simple sugerido por la guía) es la misma construcción que usa Bitcoin para el hash de bloque; es un diseño *al menos* tan fuerte frente a ataques de extensión de longitud, y no cambia ninguna propiedad observable exigida por la guía (unicidad, determinismo, avalancha). Los campos `nonce` y `dificultad` son siempre 0 en bloques PoS, y `proponente` es el minero ganador en PoW o el validador sorteado en PoS: un único formato de encabezado sirve a ambos modos.

Cada transacción (una aprobación de un revisor humano) se firma con ECDSA sobre P-256 y se verifica con la llave pública; el `tx_id` se calcula como $\text{SHA256}(\text{canónico} \parallel \text{firma})$, de modo que no puede falsificarse sin romper la firma. Un bloque debe contener *al menos una* transacción (si el *mempool* está vacío, la ronda se rechaza antes de iniciar, con mensaje claro).

Cada uno de los N nodos mantiene su propia copia (en memoria) de la cadena. Tras sellarse un bloque, éste se *difunde* a cada nodo; cada nodo acepta el bloque **si y solo si** enlaza con lo que ya tiene (su `hash_anterior` coincide con el *tip* almacenado), y lo rechaza —quedando desincronizado hasta una resincronización explícita— en caso contrario. Esto modela exactamente la regla de validación/difusión de la sección 2.4 de la guía sin requerir una red P2P real.

3. Proof of Work

Partición de nonces. El minero $i \in \{0, \dots, N-1\}$ prueba únicamente los nonces de su clase de residuo módulo N : $i, i+N, i+2N, \dots$

Lema 1 (Disyunción de clases de residuo). *Si $i_1 \neq i_2$ con $0 \leq i_1, i_2 < N$, entonces para todo $k_1, k_2 \in \mathbb{N}$, $i_1 + k_1N \neq i_2 + k_2N$.*

Demostración. Si $i_1 + k_1N = i_2 + k_2N$ entonces $i_1 - i_2 = (k_2 - k_1)N$, es decir $i_1 \equiv i_2 \pmod{N}$. Como $0 \leq i_1, i_2 < N$, la única solución es $i_1 = i_2$, contradiciendo la hipótesis. ■

Esto garantiza que **ningún dos mineros prueban jamás el mismo nonce**, y por lo tanto el valor de nonce ganador, si existe, nunca puede repetirse entre dos mineros: la regla de desempate “**gana el nonce más pequeño**” está bien definida (no hay empates reales en el valor, solo en el *momento* en que dos mineros encuentran, cada uno, su propio nonce válido dentro de la misma ronda de verificación).

Costo esperado. Modelando cada hash como una variable aproximadamente uniforme en $\{0, 1\}^{256}$ (heurística estándar de funciones hash criptográficas), la probabilidad de que un intento cumpla “ d ceros hexadecimales iniciales” es $p = 16^{-d}$. El número de intentos hasta el primer éxito sigue una distribución geométrica con $\mathbb{E}[\text{intentos}] = 1/p = 16^d$. Con N mineros trabajando en paralelo sobre clases disjuntas, el trabajo total esperado sigue siendo 16^d , pero el tiempo de pared esperado se divide aproximadamente por N . Esto justifica la cota de seguridad de 2×10^6 intentos por minero implementada: para $d \leq 6$, $16^6 \approx 1,68 \times 10^7$, cubierto con margen por $N \geq 10$ mineros en paralelo; si ningún minero resuelve dentro de ese límite, la ronda termina con **no_quorum** y un mensaje claro en vez de bloquear la aplicación indefinidamente (caso de la sección 5: “dificultad que no se resuelve en tiempo razonable”).

Madurez de la recompensa (6 confirmaciones). La recompensa del bloque de altura h se marca **confirmed** solo cuando la altura de la cadena alcanza $h+6$. La justificación es la misma que en Bitcoin: cada confirmación adicional reduce exponencialmente la probabilidad de que ese bloque sea reemplazado por una reorganización, bajo el supuesto de que ningún atacante controla una fracción sostenida de la potencia de cómputo; 6 es el valor convencionalmente aceptado para considerar el riesgo residual despreciable en la práctica.

4. Proof of Stake

Sorteo ponderado. Dado el conjunto de validadores restantes con apuestas $a_1, \dots, a_k$ y $A = \sum_j a_j$, se calcula una semilla determinística $s = \text{SHA256}(\text{hash_anterior}|\text{altura}|\text{intento})$, se interpreta como entero $r = s \bmod A$, y se recorre la lista acumulando $\text{acum} = \sum_{j \leq i} a_j$ hasta encontrar el primer i con $r < \text{acum}_i$.

Proposición 1. *Bajo el supuesto de que $s \bmod A$ se distribuye aproximadamente uniforme en $\{0, \dots, A-1\}$ (razonable para una salida de SHA-256 frente a A mucho menor que 2^{256}), el validador i es elegido con probabilidad exactamente a_i/A .*

Demostración. El validador i es elegido exactamente cuando r cae en el intervalo $[\text{acum}_{i-1}, \text{acum}_i)$, de longitud a_i . Bajo la uniformidad supuesta, $\Pr[\text{elegido} = i] = a_i/A$. ■

El parámetro **intento** (incrementado en cada redraw tras un rechazo) garantiza que un sorteo repetido con el mismo **hash_anterior** y **altura** no produzca siempre el mismo resultado.

Lema del umbral de quórum. La guía exige aceptar el bloque si $3V \geq 2A$, con V el stake que vota a favor.

Lema 2 (Equivalencia entera del umbral $2/3$). *Para $V, A \in \mathbb{Z}_{\geq 0}$, $3V \geq 2A \iff V \geq \left\lceil \frac{2A}{3} \right\rceil$.*

Demostración. ($\Leftarrow$) Si $V \geq \lceil 2A/3 \rceil \geq 2A/3$, entonces $3V \geq 2A$. ($\Rightarrow$) Si $3V \geq 2A$ entonces $V \geq 2A/3$; como V es entero, $V \geq \lceil 2A/3 \rceil$. ■

La implementación calcula $\lceil 2A/3 \rceil = \lfloor (2A+2)/3 \rfloor$ con aritmética entera (verificado exhaustivamente en `ledger_selftest` para $A \in [1, 300]$), evitando errores de redondeo de punto flotante en la condición de aceptación —incluyendo el caso límite exacto que pide la sección 5 (“votación exactamente en $2/3$ ”).

Cota de tolerancia bizantina. El umbral $\geq 2/3$ es el mínimo que garantiza que dos quóruns distintos para bloques *en conflicto* a la misma altura sean imposibles si la fracción de stake deshonesto es $< 1/3$: dos conjuntos de stake honesto, cada uno de tamaño $\geq 2A/3$, se solapan en al menos $2A/3 + 2A/3 - A = A/3$ de stake honesto; si el stake deshonesto fuera $\geq A/3$ podría, en el peor caso, estar en ambos “quóruns” simultáneamente y firmar ambas propuestas, rompiendo la seguridad. Con $< 1/3$ deshonesto esto es imposible, de ahí la cota clásica de los protocolos BFT.

Castigo y redraw. Ambas reglas de castigo de la guía están implementadas y son seleccionables por el usuario en cada ronda (parámetro `punishment_rule` de `POST /api/consensus/propose`): regla (A) $c = a_p$ (el proponente deshonesto pierde toda su apuesta y queda inhabilitado permanentemente) y regla (B) $c = \lceil \alpha \cdot a_p \rceil$, con $0 < \alpha \leq 1$ configurable — al no existir un “valor de las transacciones” monetario en este dominio sobre el cual acotar la pérdida, α se aplica directamente como el porcentaje de su propia apuesta que pierde, que es justamente la alternativa que describe la guía en prosa (“pierde su apuesta completa o un porcentaje”); bajo la regla (B) el nodo conserva el resto de su stake y sigue elegible en rondas futuras. Ambas reglas mantienen el valor esperado de proponer deshonestamente < 0 mientras el proponente tenga stake positivo en riesgo. Tras el castigo (cualquiera de las dos reglas), el nodo se remueve del conjunto elegible *de esta ronda* y se repite el sorteo con un `intento` incrementado.

Proposición 2 (Terminación del redraw). *El proceso de sorteo-rechazo-castigo termina en a lo más N intentos.*

Demostración. El conjunto de validadores elegibles pierde exactamente un elemento (el proponente deshonesto) en cada rechazo, y nunca gana elementos; es un entero no negativo estrictamente decreciente acotado por N , así que llega a 0 (“no_quorum, pool agotado”) en a lo más N pasos si nunca se acepta un bloque antes. ■

Este es precisamente el caso de la sección 5 “todos los validadores castigados hasta quedar sin saldo”: el sistema termina con un mensaje claro en vez de ciclar indefinidamente.

5. Decisiones de diseño

Sin saldo transferible ni doble gasto. El dominio de la aplicación (responsabilidad sobre trabajo de IA) no necesita mover valor entre cuentas; **stake** (PoS) y **recompensa** (PoW) son contadores internos —ajustables por API o por madurez de confirmaciones— pero nunca se transfieren de un nodo a otro, así que no existe la noción de “gastar dos veces” el mismo valor.

Copias de cadena en memoria, Postgres como referencia duradera. La guía permite simular la red en memoria. Postgres guarda la cadena canónica (persistente, auditable); cada uno de los N nodos tiene además un espejo en memoria (altura, hash) que, al recibir la difusión de un bloque nuevo, vuelve a correr *la validación completa* contra su propia copia (`Ledger::check_block`: recalcula el hash, re-verifica la regla de consenso —dificultad en PoW, sorteo y quórum en PoS— y cada firma) usando su propio `hash_anterior` almacenado como predecesor esperado, y solo adopta el bloque si esa revalidación resulta limpia. Un nodo cuya copia fue corrompida (demo de *tamper*) falla específicamente el enlace de `hash_anterior` y queda desincronizado hasta una resincronización explícita, sin necesidad de una pila de red real.

Semilla determinística y reproducible. Tanto el sorteo de PoS como la verificación de la dificultad en PoW dependen únicamente de datos ya en la cadena (`hash_anterior`, `altura`, `intento`) más la tabla de apuestas vigente; dado el mismo estado inicial, la simulación es reproducible, lo que facilita depuración y evaluación tal como pide la guía.

6. Casos contemplados y pruebas

La tabla siguiente resume cómo se cubre la lista mínima de casos de la sección 5 de la guía.

Ámbito	Cobertura
Entradas inválidas	<code>api.cpp</code> rechaza $N \notin [10, 20]$, dificultad fuera de $[1, 6]$, nodo inexistente y JSON mal formado con <code>HttpError</code> (400/404), nunca un 500; cubierto en <code>scripts/casos.mjs</code> .
Transacciones	Firma alterada / de otra llave / sobre trabajo LOW: rechazadas por <code>Ledger::check_block</code> y la ruta <code>/api/transactions</code> (<code>scripts/e2e.mjs</code>). Mempool vacío: rechazado antes de iniciar la ronda.
Cadena	<code>verify_chain</code> recalcula cada hash y liga; un bloque manipulado invalida todo lo posterior (<code>/api/demo/tamper + casos.mjs</code>).
PoW	Nonces disjuntos probados en <code>ledger_selftest</code> (Lema 1); dos ganadores resueltos por nonce mínimo; límite de intentos; recompensa a 6 confirmaciones probada minando 10 bloques en <code>casos.mjs</code> .
PoS	Sorteo ponderado y umbral 2/3 probados en <code>ledger_selftest</code> (incluyendo el límite exacto); proponente deshonesto $\rightarrow$ rechazo $\rightarrow$ castigo (reglas A y B, configurables) $\rightarrow$ redraw probado en <code>e2e.mjs</code> ; ningún validador con stake y apuesta negativa cubiertos en <code>casos.mjs</code> .
Aplicación	Reinicio de la simulación (<code>/api/demo/reset</code>), ronda ya corriendo (409), copia de nodo corrupta rechazada por revalidación independiente y resincronización cubiertos en <code>casos.mjs</code> ; bitácora unificada de eventos (<code>GET /api/events</code>) visible en la interfaz; recarga a mitad de ronda y pestañas concurrentes se verificaron manualmente (el estado vive en el servidor, no en el cliente).